

THE MICROPROCESSORS & MICROCONTROLLERS

Instructor: The Tung Than

Student: Do Quoc Huy

Student'code: 23520604

PRACTICE EXERCISE #5:

ADDITION OF TWO 32-BIT NUMBERS ON THE 8086 PROCESSOR

I. Student preparation

- Students install the [emu8086 software](#)

II. Practice content

1. Refer to the example:
 - Students open File >> example >> add/subtract
 - Run test, based on the instruction set describing its operation.
(Help >> Documentations and tutorials or Press F1)
 - Know how to reuse their screen print code
2. Write a program that adds 2 32-bit numbers.

III. Exercise

Write a program to subtract 2 32-bit numbers.

IV. Report

Compress design files and report files into a file named as follows:

[<LAB...>]-[<Student code>]-Full name

The required report file contains the following contents:

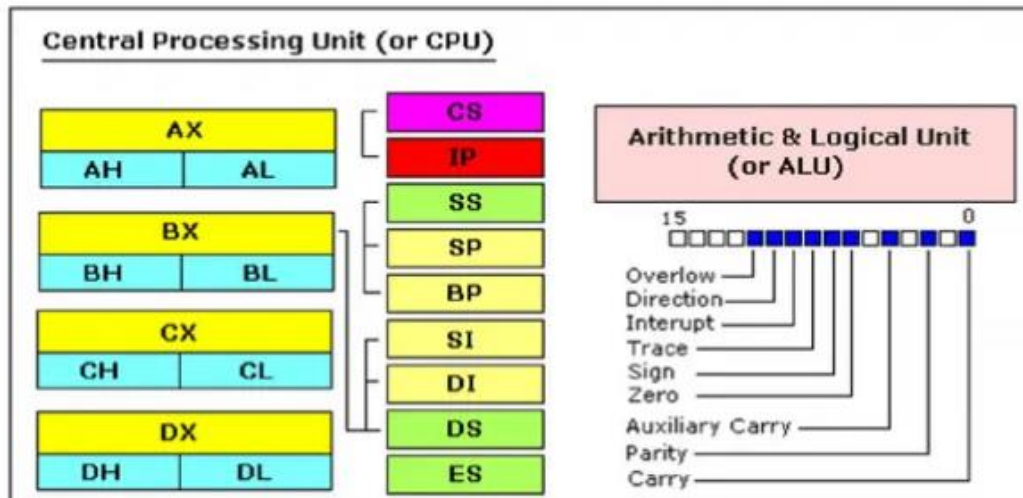
1. Describe how sample code works

- Cấu trúc của một chương trình mẫu:

Thành phần	Ý nghĩa
.model small	Khai báo mô hình bộ nhớ nhỏ
.stack 100h	Khởi tạo stack
.data	Vùng dữ liệu chứa các biến
.code	Vùng chứa mã lệnh chính
main:	Nhãn bắt đầu chương trình
int 21h	Gọi các dịch vụ của DOS để kết thúc chương trình hoặc in chuỗi

- Chức năng các thanh ghi trong 8086:

Thanh ghi	Sử dụng trong
AX	MUL, IMUL (toán hạng nguồn kích thước word) DIV, IDIV (toán hạng nguồn kích thước word) IN (nhập word) OUT (xuất word) CWD Các phép toán xử lý chuỗi (string)
AL	MUL, IMUL (toán hạng nguồn kích thước byte) DIV, IDIV (toán hạng nguồn kích thước byte) IN (nhập byte) OUT (xuất byte) XLAT AAA, AAD, AAM, AAS (các phép toán ASCII) CBW (đổi sang word) DAA, DAS (số thập phân) Các phép toán xử lý chuỗi (string)
AH	MUL, IMUL (toán hạng nguồn kích thước byte) DIV, IDIV (toán hạng nguồn kích thước byte) CBW (đổi sang word)
BX	XLAT
CX	LOOP, LOOPE, LOOPNE Các phép toán string với tiếp đầu ngữ REP
CL	RCR, RCL, ROR, ROL (quay với số đếm byte) SHR, SAR, SAL (dịch với số đếm byte)
DX	MUL, IMUL (toán hạng nguồn kích thước word) DIV, IDIV (toán hạng nguồn kích thước word)



- Bản chất của thanh ghi là ô nhớ để lưu trữ dữ liệu
- Gồm 14 thanh ghi chia làm 3 nhóm:
 - o **8 thanh dùng cho mục đích chung (general purpose)**

AX	The accumulator r (AH,AL)	SI	Source index r
BX	The base address r	DI	Destination index r
CX	The count r	BP	Base pointer
DX	The data r	SP	Stack pointer

- Các thanh ghi này có thể được dùng để chứa các 1 giá trị số tùy ý
- Kích thước mỗi thanh 16 bit
- Các thanh ghi AX, BX, CX, DX là 16 bit có thể chia đôi (vd: AL, AH) mỗi thanh ghi con 8 bit
- SI, DI, BP là các thanh ghi đặc biệt dùng để truy cập ô nhớ trong bộ nhớ

o **4 thanh segment (phân đoạn)**

CS	Trỏ tới địa chỉ segment chứa program
DS	Trỏ tới segment chứa variables được định nghĩa
ES	Extra segment r

- Code mẫu:

```

01 name "add-sub"
02
03 org 100h
04
05 mov al, 5      ; bin=00000101b
06 mov bl, 10     ; hex=0ah or bin=00001010b
07
08 ; 5 + 10 = 15 (decimal) or hex=0fh or bin=00001111b
09 add bl, al
10
11 ; 15 - 1 = 14 (decimal) or hex=0eh or bin=00001110b
12 sub bl, 1
13
14 ; print result in binary:
15 mov cx, 8
16 print: mov ah, 2 ; print function.
17         mov dl, '0'
18         test bl, 10000000b ; test first bit.
19         jz zero
20         mov dl, '1'
21 zero:   int 21h
22         shl bl, 1
23 loop print
24
25 ; print binary suffix:
26 mov dl, 'b'
27 int 21h
28
29 ; wait for any key press:
30 mov ah, 0
31 int 16h
32
33 ret
34

```

- Trong code mẫu đã chú thích kĩ từng lệnh :

○ Lệnh print:

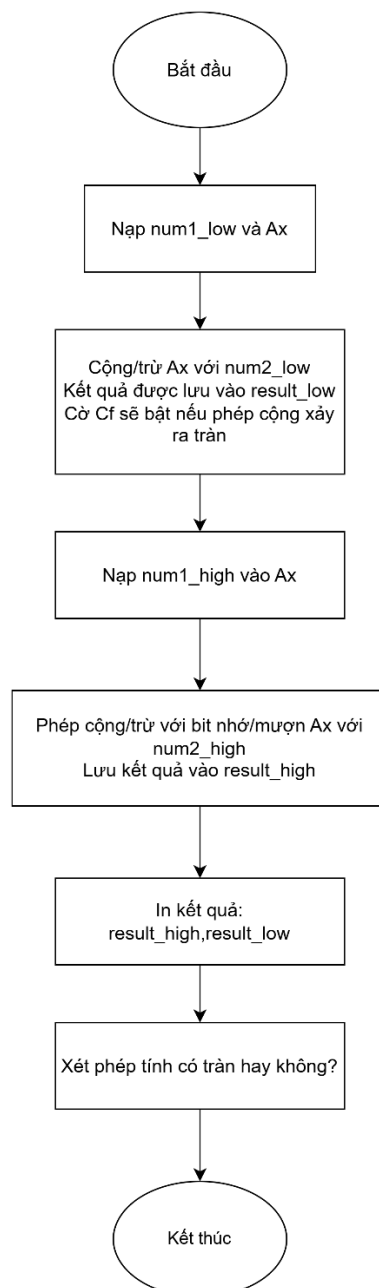
- Gán count (cx) = 8, lặp 8 lần để in 8 bit
- mov ah, 2 : chức năng in ký tự
- Gán dl = 0
- Test bl, 10000000b: kiểm tra bit đầu tiên bằng cách sử dụng thanh ghi bl AND với 10000000b, kết quả được hiển thị tại cờ ZF (zero flag)
- jz zero: Nhảy đến nhãn zero nếu cờ ZF = 1
- mov dl, '1': gán dl = 1 nếu cờ ZF = 0
- int 21h : in dl ra màn hình

- shl bl,1: dịch trái thanh ghi bl để xét các bit tiếp theo
- Sau khi in 8bit kết quả thì in thêm kí tự 'b'
- Lệnh chờ kết thúc: đọc phím nhấn
 - int 16h: ngắt BIOS được sử dụng để cung cấp các dịch vụ liên quan đến bàn phím, tùy vào giá trị thanh ghi ah mà quyết định thực hiện thao tác nào.

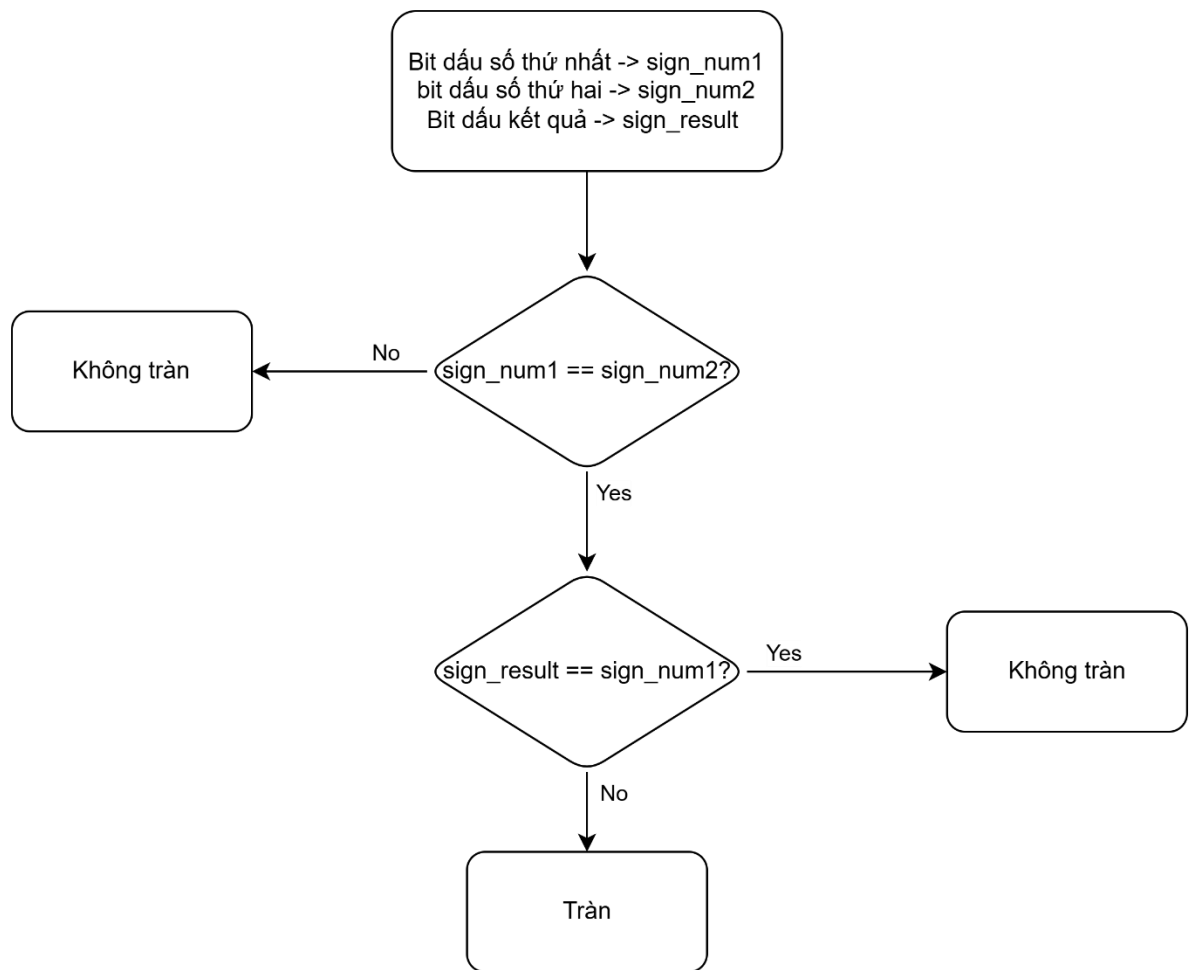
```
mov ah, 0
int 16h
```

2. Flowchart of the program algorithm to add two 32-bit numbers.

- Sơ đồ giải thuật cho bài toán:



- Sơ đồ kiểm tra tính tràn (overflow):



-

3. Explain how the algorithm works, accompanied by a video (send a Google Drive link) to demonstrate the circuit operation in case the instructor cannot run the design file.

Link drive:

<https://drive.google.com/drive/folders/1yPwSQZeIUtRiw2JIjmNTOCfDr1tx7EyO?usp=sharing>

- Code:

```

.model small
.stack 100h
.data
    ; Nhập 2 số 32-bit
    num1_low dw 1110011001100110b
  
```

```

num1_high dw 10000000000000001b

num2_low  dw 10000000000000011b
num2_high dw 1000000000000000b

; kq c?ng
sum_low   dw ?
sum_high  dw ?

; Kq tru
diff_low  dw ?
diff_high dw ?

newline db 13, 10, '$'
msg_sum  db 'Tong cua 2 so: ', '$'
msg_diff db 'Hieu cua 2 so: ', '$'
msg_overflow db 'tran, ket qua sai', 13, 10, '$'
msg_no_overflow db 'khong tran, ket qua dung', 13, 10, '$'

.code
main:
    mov ax, @data
    mov ds, ax

;=====
; TÍNH ToNG
;=====

    mov ax, num1_low
    add ax, num2_low

```

```
mov sum_low, ax
```

```
mov ax, num1_high
```

```
adc ax, num2_high
```

```
mov sum_high, ax
```

```
; In "Tong:"
```

```
lea dx, msg_sum
```

```
mov ah, 09h
```

```
int 21h
```

```
; In kq c?ng
```

```
mov cx, sum_high
```

```
call print_binary_16
```

```
mov cx, sum_low
```

```
call print_binary_16
```

```
lea dx, newline
```

```
mov ah, 09h
```

```
int 21h
```

```
; Kiem tra tràn cong
```

```
mov ax, num1_high
```

```
mov bx, num2_high
```

```
mov cx, sum_high
```

```
call check_overflow
```

```
;=====
```

```
; TÍNH HIỆU
```

```
;=====
```



```

mov ax, num1_low
sub ax, num2_low
mov diff_low, ax

mov ax, num1_high
sbb ax, num2_high
mov diff_high, ax

; In "Hieu:"
lea dx, msg_diff
mov ah, 09h
int 21h

; In kq tru
mov cx, diff_high
call print_binary_16
mov cx, diff_low
call print_binary_16
lea dx, newline
mov ah, 09h
int 21h

; Ki?m tra tr n tru
; dao dau num2
mov ax, num2_low
not ax
add ax, 1
mov si, ax      ; neg_num2_low

```

```

    mov ax, num2_high
    not ax
    adc ax, 0
    mov di, ax      ; neg_num2_high

    mov ax, num1_high ; num1_high
    mov bx, di      ; -num2_high
    mov cx, diff_high
    call check_overflow

;=====
; end
;=====

    mov ah, 4ch
    int 21h

;=====
; IN NHÌ PHÂN 16-BIT
;=====

print_binary_16 proc
    pusha
    mov bx, 16
next_bit:
    shl cx, 1
    jc print_1
    mov dl, '0'
    jmp print_do
print_1:
    mov dl, '1'

```

```

print_do:
    mov ah, 02h
    int 21h
    dec bx
    jnz next_bit
    popa
    ret
print_binary_16 endp

;=====
; KIỂM TRA TRÀN CÓ DẤU 32-BIT
; INPUT:
;  ax = số 1 high
;  bx = số 2 high
;  cx = kq high
;=====

check_overflow proc
    pusha

    ; Lấy dấu số 1
    test ax, 8000h
    mov dl, 0
    jz sign1_done
    mov dl, 1
sign1_done:

    ; Lấy dấu số 2
    test bx, 8000h
    mov dh, 0

```

```

    jz sign2_done
    mov dh, 1
sign2_done:

    ; khác dấu => không tràn
    cmp dl, dh
    jne print_no_overflow

    ; Lấy dấu kết quả
    test cx, 8000h
    mov ch, 0
    jz signr_done
    mov ch, 1
signr_done:

    ; So sánh dấu input và kết quả
    cmp dl, ch
    jne print_overflow

print_no_overflow:
    lea dx, msg_no_overflow
    mov ah, 09h
    int 21h
    jmp end_check

print_overflow:
    lea dx, msg_overflow
    mov ah, 09h
    int 21h

```

```

end_check:
    popa
    ret
check_overflow endp

end main

```

- Giải thích:

Đoạn lệnh	Chức năng
- Phần khởi tạo: <pre> .model small .stack 100h .data ; Nhập 2 số 32-bit num1_low dw 1110011001100110b num1_high dw 1000000000000001b num2_low dw 10000000000000011b num2_high dw 1000000000000000b ; Kq cộng sum_low dw ? sum_high dw ? ; Kq trừ diff_low dw ? diff_high dw ? newline db 13, 10, '\$' msg_sum db 'Tong cua 2 so: ', '\$' msg_diff db 'Hieu cua 2 so: ', '\$' msg_overflow db 'tran, ket qua sai', 13, 10, '\$' msg_no_overflow db 'khong tran, ket qua dung', 13, 10, '\$' </pre>	- Khai báo các biến num1 , num2 và kết quả kiểu dw (32 bit) để thực hiện phép cộng/trừ - In ra thông báo kết luận: tràn hay không tràn.
- Phần tính toán:	- Bởi vì độ dài các thanh ghi trong 8086 chỉ là 16 bit nên ta sẽ chia nửa toán hạng cộng/trừ với nhau - Bit nhớ sau khi thực hiện phép cộng/trừ của 2 số 16 bit thấp sẽ được lưu tại cờ CF (Carry Flag) - Ở giai đoạn cộng 16 bit cao ta sử dụng lệnh adc (add with carry) để thực hiện phép cộng cùng với bit nhớ của phép tính cộng 2 số 16 bit thấp để lại. - Ở giai đoạn trừ 16 bit cao ta sử

```

.code
main:
    mov ax, @data
    mov ds, ax

;=====
; TÍNH TỔNG
;=====
    mov ax, num1_low
    add ax, num2_low
    mov sum_low, ax

    mov ax, num1_high
    adc ax, num2_high
    mov sum_high, ax

; In "Tổng:"
    lea dx, msg_sum
    mov ah, 09h
    int 21h

; In kq c?ng
    mov cx, sum_high
    call print_binary_16
    mov cx, sum_low
    call print_binary_16
    lea dx, newline
    mov ah, 09h
    int 21h

; Kiểm tra tràn cộng
    mov ax, num1_high
    mov bx, num2_high
    mov cx, sum_high
    call check_overflow

```

dùng lệnh **sbb** (sub with borrow) để thực hiện phép trừ cùng với bit mượn của phép tính cộng 2 số 16 bit thấp để lại.

- Sau khi thực hiện xong phép cộng 2 số 32 bit, ta sẽ in ra màn hình kết quả.
- Sau khi in chúng ta kiểm tra xem kết quả có bị tràn của 2 phép tính hay không.
- Sử dụng lệnh **lea** để truy cập đến địa chỉ biến được khai báo.
- Lưu ý: ở phép trừ sau khi trừ xong, ta phải kiểm tra dấu của -num2. Vì $\text{num1} - \text{num2} = \text{num1} + (-\text{num2})$. Sau đó so sánh bit dấu của num1 và (-num2) thì mới kết luận chính xác.

```

;=====
; TÍNH HIỆU
;=====

mov ax, num1_low
sub ax, num2_low
mov diff_low, ax

mov ax, num1_high
sbb ax, num2_high
mov diff_high, ax

; In "Hieu:"
lea dx, msg_diff
mov ah, 09h
int 21h

; In kq tru
mov cx, diff_high
call print_binary_16
mov cx, diff_low
call print_binary_16
lea dx, newline
mov ah, 09h
int 21h

; Kiểm tra tràn tru
; dấu của num2
mov ax, num2_low
not ax
add ax, 1
mov si, ax      ; n

```

- Hàm in ra màn hình:

- Lệnh ***pusha*** (push all) dùng để lưu tất cả thanh ghi tổng quát vào stack theo thứ tự: AX → CX → DX → BX → SP → BP → SI → DI
- Lệnh ***popa*** (pop all) dùng để khôi phục các thanh ghi từ stack **ngược thứ tự** so với

<pre> ;----- ; In 16-bit nhị phân tu CX ;----- print_binary_16 proc pusha mov bx, 16 next_bit: shl cx, 1 jc print_1 mov dl, '0' jmp print_do print_1: mov dl, '1' print_do: mov ah, 02h int 21h dec bx jnz next_bit popa ret print_binary_16 endp end main </pre>	pusha - Cũng tương tự thực hiện phép tính cộng / trừ , thao tác in ra cũng phải chia nửa, mov bx,16 dùng để in ra 16 lần , mỗi lần 1 chữ số. - shl cx, 1 : Dịch trái (shift left) thanh ghi CX 1 bit. Bit cao nhất (MSB) của CX sẽ được đẩy vào cờ nhớ (Carry Flag - CF) - Nếu CF = 1 (bit vừa dịch =1), nhảy đến nhãn print_1. Ngược lại, thực hiện lệnh sau. - Sử dụng thanh ghi dl để xuất chữ số. - Trong hàm print_do gọi ngắt 21h với thanh ghi ah có giá trị là 02h để xuất kí tự và trừ biến đếm 1 đơn vị. - jnz next_bit: Nếu bx = 0 thì thoát vòng lặp. Ngược lại tiếp tục chạy. - popa: Khôi phục lại các thanh ghi đã lưu từ stack.
- Kiểm tra tràn:	- Sử dụng các thanh ghi để lưu bit dấu của các toán hạng và kết quả. - ax = num1 high - bx = num2 high - cx = kq high - dl = sign(num1) - dh = sign(num2) - check_overflow proc: gọi thủ


```

check_overflow proc
    pusha

    ; Lay dau so 1
    test ax, 8000h
    mov dl, 0
    jz sign1_done
    mov dl, 1
sign1_done:

    ; Lay dau so 2
    test bx, 8000h
    mov dh, 0
    jz sign2_done
    mov dh, 1
sign2_done:

    ; khac dau => không tràn
    cmp dl, dh
    jne print_no_overflow

    ; Lay dau ket qua
    test cx, 8000h
    mov ch, 0
    jz signr_done
    mov ch, 1
signr_done:

    ; So sánh dau input và ket
    cmp dl, ch
    jne print_overflow

```

tục kiểm tra overflow.

- Thuật toán kiểm tra tràn đã được nêu ở phần trên.
- **cmp dl, dh:** So sánh dấu của AX và BX. Nếu khác dấu → không tràn và ngược lại có thể tràn.
- **cmp dl, ch:** So sánh dấu của input (DL) và kết quả (CH). Lúc này ta sẽ so sánh với dấu của kết quả xem có tràn hay không. Nếu cùng dấu thì không tràn và ngược lại.
- Sau khi kiểm nghiệm xong thì xuất thông báo.

```

print_no_overflow:
    lea dx, msg_no_overflow
    mov ah, 09h
    int 21h
    jmp end_check

print_overflow:
    lea dx, msg_overflow
    mov ah, 09h
    int 21h

end_check:
    popa
    ret
check_overflow endp

end main

```

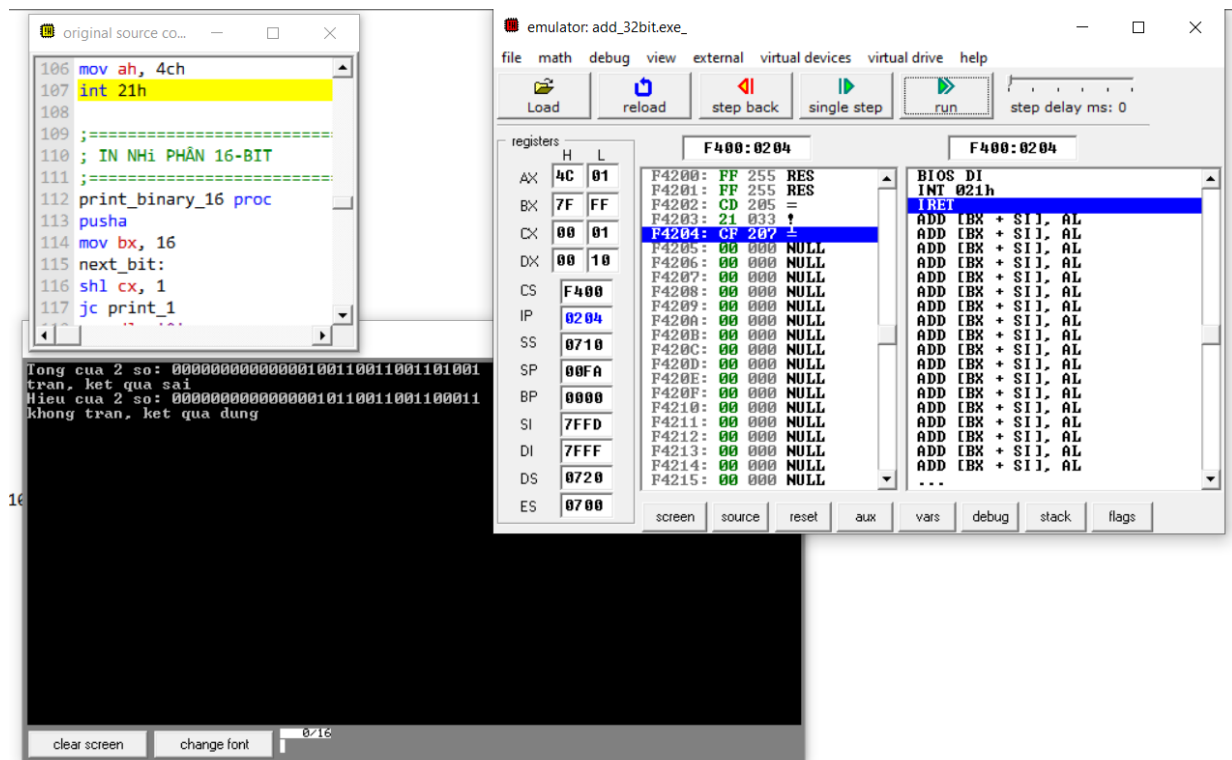
- Kết quả sau khi chạy chương trình:

emulator screen (80x25 chars)

```

Tong cua 2 so: 0000000000000000100110011001101001
tran, ket qua sai
Hieu cua 2 so: 000000000000000010110011001100011
khong tran, ket qua dung

```



⇒ Chương trình đã chạy đúng theo yêu cầu của đề bài

V. Reference

- [Syslabus – Phan Duy](#)
- [KyThuatLapTrinhNhungs- ngo huu huy.pdf](#)
- [Chương 5: Giới thiệu VXL 8086](#)
- ChatGPT